Step 3 — Trace One Instruction Completely
Instruction: add t2, t0, t1 (PC = 0x00000008, t0 = 12, t1 = 5)

Element	Port	Value
pc_reg	out	**0x00000008**
pc_4	out	0x0000000C
instr_mem	data_out	**0x006283B3**
decode	r1_reg_idx / r2_reg_idx	5 / 6
decode	wr_reg_idx	**7**
control	reg_do_write_ctrl	1
control	alu_op2_ctrl	0 (register, not immediate)
registerFile	r1_out / r2_out	**12 / 5**
alu_op1_src	out	12
alu_op2_src	out	5
alu	res	17
data_mem	wr_en	0
reg_wr_src	select / out	0 / 17 (ALU result)
pc_src	select / out	0 / 0x0000000C (PC+4)

Step 4 — Trace Every Instruction Class
One row per RV32I class (values observed on the Extended single-cycle datapath).

Instruction	Format	Imm	op1 mux	op2 mux	alu.res	wb mux / pc mux
addi t0, zero, 12	I	12	reg (0)	imm (1)	12	ALU / PC+4
add t2, t0, t1	R	—	reg (0)	reg (0)	17	ALU / PC+4
lui t4, 0x2B	U	0x2B000	0 (lui)	imm	0x2B000	ALU / PC+4
auipc t5, 0x0	U	0	PC	imm	0x14 (20)	ALU / PC+4
lw a1, 0(a0)	I	0	reg	imm	addr	Mem / PC+4
sw t2, 4(a0)	S	4	reg	imm	0x10000004	— (no WB) / PC+4
beq t0, t1, skip	B	offset	reg	reg	cmp	— / PC+4 (not taken)
bne t0, t1, target	B	offset	reg	reg	cmp	— / target (taken)
jal ra, report	J	offset	—	—	—	PC+4 / target
jalr zero, ra, 0	I	0	reg (ra)	imm	target	PC+4 / ALU

Step 4 — Answers to the four questions

1. **lui vs auipc:** They share the U-type format and the same immediate. The difference is the ALU operand-1 multiplexer: lui forces a zero, auipc selects the current PC. That single select bit produces the two different results.
2. **sw never writes a register:** The single control signal is MemWrite (data_mem.wr_en = 1). The write-back multiplexer is unused / don't-care for stores; RegWrite is de-asserted.
3. **beq vs bne:** Both compare the same two registers in the Branch unit. The equality (or inverted equality) result travels through the control-flow OR gate to the PC-source multiplexer select. Only the select value differs.
4. **jal / jalr return address:** Supplied by the third input of the write-back multiplexer (PC+4). Neither the ALU result nor the data-memory output can produce the sequential next-PC value required for the link register.

Numerical Answers

#	Question	Answer
Q1	trace-classes.s: how many instructions does the Instruction memory panel list?	**18**
Q2	trace-classes.s: how many of those are never executed?	**1**
Q3	Single-cycle processor: the CPI reported in the Statistics panel	**1.00**

Q4	Value of t4 after lui t4, 0x2B, in decimal	**176128**
Q5	Value of t5 after auipc t5, 0x0, in decimal	**20**
Q6	The 32-bit encoding of add t2, t0, t1	**006283B3**
Q7	The value on the immediate generator output while sw t2, 4(a0) executes	**4**
Q8	The value on alu.res while sw t2, 4(a0) executes, in decimal	**268435460**
Q9	The value on the branch unit output while beq t0, t1, skip executes	**0**
Q10	The value on pc_src.select while bne t0, t1, target executes	**1**

Step 6 — Implementation vs Textbook Diagram
Three substantive differences and the engineering gain of each:

#	What the Week 6 figure shows	What the implementation does & the gain
A	Dedicated branch adder, separate from the ALU, computing PC + offset	No second adder. Branch target is computed by the main ALU; a multiplexer selects the result. Hardware saving (one fewer adder) at the cost of a slightly longer critical path when a branch is taken.
B	beq tested by the ALU's Zero flag after a subtraction	Separate comparison / Branch unit. It can test equality, inequality and the signed/unsigned relations that a simple Zero flag cannot express, and it frees the ALU for other work on the same cycle.
C	Two-input MemtoReg multiplexer	Three-input write-back multiplexer (ALU result / memory data / PC+4). The third input is required by jal and jalr so that the return address can be written without an extra adder in the write-back path.